

Orientación a objetos

Encapsulación

Imaginemos que escribimos el código para una clase, y que una docena de programadores de nuestra compañía escriben programas que usan dicha clase. Ahora imaginemos que más tarde, no nos gusta la manera en la que funciona nuestra clase porque al ser usada dentro del código de los otros programadores algunas variables de instancia reciben valores que nosotros no habíamos previsto: El código de los demás programadores provoca errores en nuestro código.

Deberíamos de ser de crear una nueva versión de la clase, la cual los demás programadores podrán reemplazar en sus programas sin necesidad de cambiar su propio código. La única posibilidad de solucionar esto es reescribir nuestra clase original. La posibilidad de hacer cambios en nuestro código sin estropear el código de otros es un beneficio clave de la **encapsulación**.

La encapsulación nos permite esconder los detalles de la implementación detrás de una API. De esta forma, escondiendo los detalles de la implementación, podemos rehacer el código de nuestro método sin forzar un cambio en el código que invoca al método que hemos cambiado. La habilidad para hacer cambios en nuestra implementación de código sin romper el código de otros que utilizan nuestro código es el beneficio de la encapsulación. La encapsulación nos proporciona mantenibilidad, flexibilidad y extensibilidad. ¿Cómo lo hacemos?

- Mantener las variables de instancia protegidas (con un modificador de acceso, a menudo private).
- Hacer métodos de acceso públicos (setters y getters), y forzar a utilizarlos en vez de permitir acceder directamente a la variable de instancia.
- Para esos métodos de acceso, usaremos la convención más común de set<Propiedad> y get<Propiedad>.
- Siempre debemos forzar al código que invoca a usar los métodos de la API en vez de que acceda directamente a las variables de instancia.

```
private int NumPag;  
private double Precio;  
  
public void setPrecio(double precio) {  
    String Cambio="" + precio;  
    String Resultado=IsNumeric(Cambio, 2);  
    if(Resultado=="Double" && precio>100) { Precio = precio; }  
    else { Precio = 00.00; }  
}
```

```

public void setNumPag(int numPag){
    String Cambio="" + numPag;
    String Resultado=IsNumeric(Cambio, 1);
    if(Resultado=="Integer" && numPag<800){
        NumPag = numPag;
    }else{
        NumPag = 0000;
    }
}
}

```

Herencia y polimorfismo

Es imposible escribir incluso el programa más pequeño en Java sin usar la herencia, ya que toda clase de Java es subclase de la clase **Object**.

- Operador ***instanceof***: Devuelve *true* si la variable de referencia testada es del tipo comparado.
- Método ***equals*** de la clase Object: Devuelve *true* si dos variables hacen referencia al mismo objeto.

```

class Test {
    public static void main(String [ ] args) {
        Test t1 = new Test( );
        Test t2 = new Test( );
        if (!t1.equals(t2))
            System.out.println("No son referencias al mismo objeto");
        if (t1 instanceof Object)
            System.out.println("t1 es un Object");
    }
}

```

La evolución de la herencia

Hasta Java 8 el asunto de la herencia giraba en torno a subclases heredando métodos de sus superclases. Esta simplificación no es muy correcta, y con Java 8 menos aún ya que ahora es posible heredar métodos concretos desde interfaces. Es un gran cambio. En lo sucesivo, cuando hablamos de subtipos y supertipos, los interfaces tendrán que ser tenidos en cuenta junto con las clases.

Elements of Types	Classes	Interfaces
Instance variables	Yes	Not applicable
Static variables	Yes	Only constants
Abstract methods	Yes	Yes
Instance methods	Yes	Java 8, default methods
Static methods	Yes	Java 8, inherited no, accessible yes
Constructors	No	Not applicable
Initialization blocks	No	Not applicable

La tabla muestra los elementos heredables por clases e interfaces.

Las dos razones más comunes para usar la herencia son:

- Promover la reutilización del código: Se crea una clase genérica con la intención de crear subclases más especializadas que heredan de esta. Todas las subclases especializadas tienen todos los métodos y atributos de la superclase que extienden.
- Usar el polimorfismo: Posibilidad de tratar cualquier objeto de una subclase como si fuese un objeto de la superclase.

Limitación del polimorfismo:

```
class Juego {
    public void mostrarJuego( ) {
        System.out.println("Esto es un juego");
    }
    // más código aquí.
}
class JuegoConFichas extends Juego {
    public void moverFicha( ) {
        System.out.println("Moviendo una ficha");
    }
    // más código aquí.
}
class Parchis extends JuegoConFichas {
    public void comerFicha( ) {
        System.out.println("Comiendo la ficha de al lado.");
    }
    // más código aquí.
}
```

```

public class Test {
    public static void main (String[] args) {
        JuegoConFichas fichaGenerica = new JuegoConFichas( );
        Parchis fichaAzul = new Parchis( );
        hacerJugada(fichaGenerica);
        hacerJugada(fichaAzul);
    }
    public static void hacerJugada(Juego jg) {
        jg.mostrarJuego();
    }
}

```

Cualquier de Juego puede ser pasado a un método con un argumento del tipo Juego. Sin embargo hay implicaciones. El método **hacerJugada** sabe sólo que los objetos son un tipo de Juego, ya que ese es el parámetro que se ha declarado. Y usando una referencia declarada como del tipo Juego, sin importar la variable es un parámetro de un método, una variable local o una variable de instancia, significa que sólo se podrán invocar los métodos de Juego. Los métodos que se pueden invocar en una referencia son totalmente dependientes del tipo de la variable declarada, sin importar el tipo de objeto de que se trate.

Relaciones Es-Un, Tiene-Un

- **ES-UN**

En orientación a objetos, el concepto Es-Un esta basado en la herencia de clases o en la implementación de interfaces. Es-Un es una forma de decir: “Esta cosa es de ese tipo de cosas”. Una relación Es-Un se expresa en Java usando las palabras clave ***extends*** (para la herencia de clases) y ***implements*** (para la implementación de interfaces).

```

public class Coche {
    // Más código aquí.
}
public class Subaru extends Coche {
    // Código específico de los Subaru.
    // No hay que olvidar que Subaru hereda todos los miembros accesibles de Coche y
    //esto puede incluir métodos y variables.
}

```

Si tenemos un árbol de herencia como este:

```

public class Vehiculo { ... }
public class Coche extends Vehiculo { ... }
public class Subaru extends Coche { ... }

```

Se cumple que:

Vehiculo es superclase de Coche.
Coche es una subclase de Vehiculo.
Coche es superclase de Subaru.
Subaru es una subclase de Vehiculo.
Coche hereda de Vehiculo.
Subaru hereda tanto de Vehiculo como de Coche.
Subaru es un subtipo de Coche.
Coche es un subtipo de Vehiculo.
Subaru es un subtipo de Vehiculo.

- **HAS-A**

Las relaciones Tiene-Un están basadas en el uso, en vez de en la herencia. En otras palabras, la clase A Tiene-Un B si el código en la clase A tiene una referencia a una instancia de la clase B (Un Caballo ES-Un Animal. Un Caballo **Tiene-Un** Percherón).

El código podría ser:

```
public class Animal { }  
public class Caballo extends Animal {  
    private Percheron miPercheron;  
}
```

Polimorfismo

En Java cualquier objeto que pueda pasar más de un test Es-Un puede ser considerado polimórfico. Es decir, excepto los objetos de la clase Object, todos los objetos son polimórficos.

Recordemos que la única manera de acceder a un objeto es a través de una variable de referencia, y debemos recordar acerca de las referencias que:

- Una variable de referencia puede ser sólo de un tipo, y una vez declarada, ese tipo ya no podrá ser cambiado (aunque cambie el objeto al que referencia).
- Una referencia es una variable, así que puede ser reasignada a otros objetos (a menos que la referencia haya sido declarada **final**).
- El tipo de una variable de referencia determina los métodos que pueden ser invocados en el objeto que la variable está referenciando.
- Una variable de referencia se puede referir a cualquier objeto del mismo tipo que la referencia declarada o, SE PUEDE REFERIR A CUALQUIER SUBTIPO DEL TIPO DECLARADO.
- Una variable de referencia puede ser declarada como un tipo de clase o un tipo de interface. **Si la variable es declarada como un tipo de interface, esta puede referenciar a cualquier objeto de cualquier clase que implemente el interface.**

Las invocaciones de métodos permitidas por el compilador están basadas únicamente **en el tipo** de la referencia declarada, sin importar el tipo de objeto.

Si la subclase sobrescribe el método invocado, se ejecuta el método de la subclase, aunque el compilador chequea el de la superclase. **Sólo los métodos sobrescritos de una instancia son dinámicamente invocados basándose en el tipo real del objeto.**

Las invocaciones a métodos polimórficos se aplican sólo a métodos de instancia. Siempre nos podemos referir a un objeto con tipo de variable de referencia más general (una superclase o un interfaz), pero en tiempo de ejecución, la ÚNICA cosa que dinámicamente seleccionada basándose en el objeto actual (en vez de en el tipo de referencia) son los métodos de instancia. No los métodos static. No las variables. **Sólo los métodos de instancia sobrescritos son dinámicamente invocados basándose en el tipo real del objeto.**

Sobreescritura / Sobrecarga

Métodos sobrescritos

Un método de instancia en una subclase con la misma signatura (nombre, más el número y tipo de sus parámetros) y tipo de retorno que un método de instancia de la superclase, **sobrescribe** el método de la superclase. El beneficio clave de sobrescribir es la posibilidad de definir un comportamiento que es específico de un subtipo particular.

```
public class Animal {
    public void comer( ) {
        System.out.println("Animal que come de todo.");
    }
}

class Caballo extends Animal {
    public void comer( ) {
        System.out.println("Caballo comiendo cosas de caballo.");
    }
}

public class TestAnimales {
    public static void main (String [] args) {
        Animal a = new Animal( );
        Animal b = new Caballo( ); //Polimorfismo.
        a.comer( ); // Ejecuta la versión Animal de comer( ).
        b.comer( ); // Ejecuta la versión Caballo de comer( ).
    }
}
```

Un método que sobrescribe también puede retornar un subtipo del tipo devuelto por el método sobrescrito. Este es llamado un tipo de retorno **covariante**.

- Un método marcado como **final**, no podrá ser sobrescrito.
- Un método marcado como **static** no puede ser sobrescrito.
- Si un método no puede ser heredado, no puede ser sobrescrito.

Para **métodos abstract** heredados de una superclase no hay elección, deberemos implementar dicho método en la subclase **a menos que la subclase también sea abstract**. Los métodos abstract deben obligatoriamente ser implementados por la primera clase no abstract que los herede. Podemos pensar en los métodos abstract como métodos que estamos obligados a sobrescribir.

El método que sobrescribe no puede tener un modificador de acceso **más restrictivo** que el método que está siendo sobrescrito (a causa del polimorfismo), pero si **menos restrictivo**.

```
public class Animal {
    public void comer( ) {
        System.out.println("Animal que come de todo.");
    }
}

class Caballo extends Animal {
    private void comer( ) {
        System.out.println("Caballo comiendo cosas de caballo.");
    }
}

public class ProbarAnimales{
    public static void main(String[ ] args){
        Animal a = new Animal( );
        Animal b = new Caballo( );
        a.comer( ); // Ejecuta la versión Animal de comer( )
        b.comer( ); // Ejecuta la versión Caballo de comer( )
    }
}
```

¿Funciona?

Las **reglas para sobrecribir un método** son las siguientes:

- La lista de argumentos debe coincidir exactamente con la del método sobrescrito.

- El tipo de retorno debe de ser el mismo, o un subtipo, del tipo retornado por el método original de la superclase sobreescrito.
- El nivel de acceso no puede ser más restrictivo que el del método sobreescrito.
- El nivel de acceso puede ser menos restrictivo que el del método sobreescrito.
- Los métodos de instancia pueden ser sobreescritos sólo si son heredados por el subtipo. Un subtipo dentro del mismo paquete que la instancia del supertipo puede sobrescribir cualquier método del supertipo que no esté marcado como **private** o **final**.
- Un subtipo en un paquete diferente puede sobrescribir sólo aquellos métodos no final marcados como public o protected (ya que los métodos protected son heredados por el subtipo).
- El método que sobrescribe puede lanzar cualquier excepción NO comprobada (runtime), sin importar si el método sobreescrito declara la excepción.
- El método que sobrescribe **NO DEBE** lanzar excepciones comprobadas que sean nuevas o más amplias que aquellas declaradas por el método sobrescrito.
- El método que sobrescribe puede menos excepciones o excepciones más restrictivas. Sólo porque el método sobreescrito toma riesgos no significa que la excepción del subtipo que sobrescribe tome los mismos riesgos. Un método que sobrescribe no tiene que declarar ninguna excepción que nunca lanzará, sin importar de que declare el método sobreescrito.
- No se puede sobrescribir un método marcado como **final**.
- No se puede sobrescribir un método marcado **static**.
- Si un método no puede ser heredado, no puede ser sobreescrito.

```
class Animal{
    private void comer( ){
        System.out.println("Animal genérico comiendo.");
    }
}
class Caballo extends Animal{ }
public class TestAnimales{
    public static void main(String[ ] args){
        Caballo c = new Caballo( );
        c.comer();
    }
}
```

¿Funciona?

Invocando la versión de la superclase de un método sobrescrito

Para ello se utiliza la palabra **super**: Se ejecutará la versión del método de la superclase, pero después vuelve y se termina con el código adicional en el método de la subclase.

```
public class Animal {
    public void comer( ) { }
    public void metodoPrueba( ) {
        // Código útil.
    }
}

class Caballo extends Animal {
    public void metodoPrueba( ) {
        // Aprovechar el código de la superclase y añadir más funcionalidades.
        super.metodoPrueba( ); // Aquí se invoca a el método de la superclase.
        // Código específico de la clase Caballo.
    }
}
```

De la misma forma, podemos acceder a un método sobreescrito de un interfaz con la sintáxis:

InterfaceX.super.nombreMetodo();

Nota: Usar **super** para invocar al método sobreescrito sólo se aplica a los métodos sobreescritos. Y sólo podemos usar **super** para acceder a un método del supertipo del tipo, es decir, **NO** podemos usar **super.super.nombreMetodo()** y **NO** podemos usar **InterfaceX.super.super.nombreMetodo()**.

Illegal Override Code	Problem with the Code
<code>private void eat() { }</code>	Access modifier more restrictive
<code>public void eat() throws IOException { }</code>	Declares a checked exception not defined by superclass version
<code>public void eat(String food) { }</code>	A legal overload, not an override, because the argument list changed
<code>public String eat() { }</code>	Not an override because of the return type, and not an overload either because there's no change in the argument list

Métodos sobrecargados

Los métodos sobrecargados permiten reutilizar el mismo nombre del método en la clase, pero con diferentes argumentos (y, opcionalmente, un tipo de retorno diferente).

Las reglas para métodos sobrecargados son:

- Los métodos sobrecargados **deben** de cambiar la lista de argumentos.
- Los métodos sobrecargados **pueden** cambiar el tipo de retorno.
- Los métodos sobrecargados **pueden** cambiar el modificador de acceso.
- Un método puede ser sobrecargado en la misma clase o en una subclase de esta.

Cuando se ejecuta el programa, la decisión de que método sobrecargado ejecutar se basa en los argumentos.

```
public class Sobrecarga {  
    public void Numeros(int x, int y){  
        System.out.println("Método que recibe enteros.");  
    }  
    public void Numeros(double x, double y){  
        System.out.println("Método que recibe flotantes.");  
    }  
    public void Numeros(String cadena){  
        System.out.println("Método que recibe una cadena: "+ cadena);  
    }  
    public static void main (String... args){  
        Sobrecarga s = new Sobrecarga();  
        int a = 1;  
        int b = 2;  
        s.Numeros(a,b);  
        s.Numeros(3.2,5.7);  
        s.Numeros("Monillo007");  
    }  
}
```

Invocando métodos sobrecargados

```
class Animal { }  
class Caballo extends Animal { }  
class TestAnimales {  
    public void hacerCosas(Animal a ) {  
        System.out.println("Caballo haciendo cosas de caballo.");  
    }  
    public void hacerCosas(Caballo b ) {  
        System.out.println("Caballo haciendo otras cosas, también de caballo.");  
    }  
}
```

```

public static void main (String [] args) {
    TestAnimales ua = new TestAnimales( );
    Animal an = new Animal( );
    Caballo cb = new Caballo();
    ua.hacerCosas(an);
    ua.hacerCosas(cb);
    Animal arefc = new Caballo( );
    ua.hacerCosas(arefc); // ¿Qué versión del método sobrecargado de invoca?
}
}

```

¿Qué versión del método sobrecargado de invoca?

Incluso aunque el objeto pasado en tiempo de ejecución es un Caballo y no un Animal, la elección de a qué método sobrecargado llamar, NO se decide dinámicamente en tiempo de ejecución.

Debemos recordar que el tipo de la referencia (no el tipo de objeto) determina que método sobrecargado es invocado.

Para resumir, que versión de un método **sobreescrito** llamar (en otras palabras, de qué clase en el árbol de herencia) se decide en tiempo de ejecución basándose en el tipo de objeto, pero que versión del método llamar en caso de **sobrecarga** se decide basándose en el tipo de referencia del argumento pasado en tiempo de compilación.

Si invocamos a un método pasándole una referencia Animal a un objeto Caballo, el compilador sólo conoce al Animal, de forma que elige la versión sobrecargada del método que toma un Animal. No importa que , en tiempo de ejecución, de hecho se pase un Caballo.

Method Invocation Code	Result
Animal a = new Animal(); a.eat();	Generic Animal Eating Generically
Horse h = new Horse(); h.eat();	Horse eating hay
Animal ah = new Horse(); ah.eat();	Horse eating hay Polymorphism works—the actual object type (Horse), not the reference type (Animal), is used to determine which eat () is called.
Horse he = new Horse(); he.eat("Apples");	Horse eating Apples The overloaded eat (String s) method is invoked.
Animal a2 = new Animal(); a2.eat("treats");	Compiler error! Compiler sees that the Animal class doesn't have an eat () method that takes a String.
Animal ah2 = new Horse(); ah2.eat("Carrots");	Compiler error! Compiler still looks only at the reference and sees that Animal doesn't have an eat () method that takes a String. Compiler doesn't care that the actual object might be a Horse at runtime.

La tabla anterior muestra ejemplos de sobreescritura legal e ilegal. La siguiente muestra las diferencias entre métodos sobrecargados y métodos sobreescritos.

	Overloaded Method	Overridden Method
Argument(s)	Must change.	Must not change.
Return type	Can change.	Can't change except for covariant returns. (Covered later this chapter.)
Exceptions	Can change.	Can reduce or eliminate. Must not throw new or broader checked exceptions.
Access	Can change.	Must not make more restrictive (can be less restrictive).
Invocation	<i>Reference</i> type determines which overloaded version (based on declared argument types) is selected. Happens at <i>compile</i> time. The actual <i>method</i> that's invoked is still a virtual method invocation that happens at runtime, but the compiler will already know the <i>signature</i> of the method to be invoked. So at runtime, the argument match will already have been nailed down, just not the <i>class</i> in which the method lives.	<i>Object</i> type (in other words, <i>the type of the actual instance on the heap</i>) determines which method is selected. Happens at <i>runtime</i> .

Casting

```

class Animal{
    void hacerRuido( ) {System.out.println("Ruido genérico");}

class Perro extends Animal{
    void hacerRuido( ) { System.out.println("LadRAR"); }
    void hacerTrucos(){ System.out.println("Hacerse el muerto."); }
}

class CastTest{
    public void main(String[ ] args){
        Animal[ ] a = { new Animal( ), new Perro( ), new Animal( ) };
        for( Animal animal : a){
            animal.hacerRuido( );
            if( animal instanceof Perro ) {
                animal.hacerTrucos();
            }
        }
    }
}

```

¿Qué ocurre?

- **Downcast** o casting hacia abajo en el árbol de herencia:

```
if ( animal instanceof Perro){  
    Perro p = (Perro) animal;  
}
```

Es importante saber que el compilador está forzado a confiar en nosotros cuando hacemos un casting, incluso aunque estemos equivocados (en ese caso se generará una excepción).

- **Upcasting** o casting hacia arriba en el árbol de herencia:

```
Perro p = new Perro( );  
Animal a1 = d; // Upcasting no explícito.  
Animal a2 = (Animal) d; // Upcasting explícito.
```

En este caso sólo se podrán invocar a los métodos de la superclase, aunque estén sobrescritos.

Implementando un interface

Cuando se implementa un **interface**, estamos acordando adherirnos al contrato definido en el interface. Esto significa que estamos acordando proveer de implementaciones legales de todos los métodos definidos en el interface, y que cualquiera que sepa que métodos incluye el interface (no como están implementados, sino como pueden ser invocados y lo que retornan), puede estar seguro de que puede invocar dichos métodos en una instancia de la clase que se esta implementando.

```
public interface Botable{  
    public void botar( );  
    public void establecerFrecuenciaDeBote(int bf);  
}  
  
public abstract class Pelota implements Botable { // Palabra clave implements.  
    public void botar( );  
    public void establecerFrecuenciaDeBote(int bf); // No es obligatorio reescribir los  
                                                // métodos abstract en la clase abstract.  
}
```

Las clases que implementan interfaces se adhieren a las mismas reglas para la implementación de métodos que una clase que extiende una clase **abstract**. Para ser una clase de implementación legal, una clase de implementación no abstract, debe de hacer lo siguiente:

- Proveer implementaciones concretas (no abstract) para TODOS los métodos declarados en el interface.
- Seguir todas las reglas legales para la sobrescritura como las siguientes:
 - Declarar las excepciones no comprobadas en los métodos de implementación que no se hayan declarado en el método de interfaz o subclases de aquellas excepciones declaradas por el método de interfaz.
 - Mantener la signatura del método del interface, y mantener el mismo tipo de retorno (o un subtipo).

```
class PelotaPlayera extends Pelota {
    // Aunque no se especifique en esta declaración,
    // PelotaPlayera implementa Botable, ya que la
    // superclase Pelota implementa Botable.
    public void botar( ) {
        // Código específico de la forma de botar de la pelota de playa.
    }

    public void establecerFrecuenciaDeBote(int bf) {
        // Estableciendo botes por segundo.
    }
    // Si la clase Pelota tuviese definidos métodos abstract
    // propios, tendrían que ser definidos aquí también.
}
```

A menos que la clase que implementa el interface sea **abstract**, la clase que lo implementa debe proveer implementaciones para todos los métodos definidos en el interface. Si la clase es abstract puede pasar problema a su primera subclase concreta.

Dos reglas más a tener en cuenta:

1. Una clase puede implementar más de un interface.

```
public class Pelota implements Botable, Pateable, Lanzable { ... }
```

2. Un interface puede extender (extends) a otro u otros interfaces, pero nunca implementar nada. A una clase sólo se le permite extender a otra como mucho, sin embargo, un interface puede extender múltiples interfaces.

```
public interface Botable extends Movable { }
```

Java 8 – Ahora con herencia múltiple

Desde Java 8 los interfaces pueden tener métodos concretos. Como además ya sabemos que las clases pueden implementar múltiples interfaces, el espectro de herencia múltiple puede ser complicado. Una clase puede implementar interfaces con métodos concretos con **signaturas duplicadas**. Si queremos implementar dos interfaces con métodos solapados tendremos que proveer un método sobrescrito en nuestra clase:

```
interface I1{
    default int dostuff( ){ return 1; }
}

interface I2{
    default int dostuff( ){ return 2; }
}

public class MultiInt implements I1,I2{
    public static void main(String[ ] args){
        new MultiInt.go( );
        void go( ){
            System.out.println(doStuff( ));
        }

        // public int doStuff( ){
        // return 3;
        // }   Si descomentamos este código compilaria, sino no.
    }
}
```

Tipos de retorno legales

Declaraciones de tipo de retorno

- **Tipos de retorno en métodos sobrecargados**

Si heredamos un método y lo sobrecargamos en un subtipo, no estamos sujetos a las restricciones de la sobrescritura. Esto significa que podemos declarar cualquier tipo de retorno que queramos. Lo que **NO** podemos hacer es cambiar únicamente el tipo de retorno. El método sobrecargado **DEBE** cambiar la lista de argumentos.

- **Sobrescritura y tipos de retorno y retornos covariantes**

Cuando un subtipo quiere cambiar la implementación de un método heredado (y sobrescrito), el subtipo debe definir un método que coincida exactamente con la

versión heredada. Desde Java 5, está permitido cambiar el tipo de retorno en el método sobrescrito en cuanto el nuevo tipo de retorno **sea un subtipo** del tipo de retorno declarado del método sobrescrito (superclase).

Retornando un valor

Hay que tener en cuenta seis reglas para retornar un valor:

1. Podemos retornar **null** en un método con un tipo de retorno que sea una referencia a un objeto.
2. Un array es un tipo de retorno perfectamente legal.
3. En un método con una primitiva como tipo de retorno, podemos retornar cualquier valor o variable que pueda ser implícitamente convertida al tipo de retorno declarado.
4. En un método con una primitiva como tipo de retorno, podemos retornar cualquier valor o variable que pueda ser explícitamente convertida (casting) al tipo de retorno declarado.
5. Un método con un tipo de retorno **void** no debe retornar nada.
6. En un método con un tipo de retorno que sea una referencia a objeto, se puede retornar cualquier tipo de que pueda ser implícitamente convertido (cast) al tipo de retorno declarado.

```
public Animal getAnimal( ){  
    return new Caballo( ); // Asumiendo que Caballo extiende a Animal.  
}  
  
public Object getObject( ){  
    int[ ] nums = { 1,2,3 };  
    return nums;  
}  
  
public interface Masticable{ }  
public class Chicle implements Masticable{ }  
  
public class TestMasticable{  
    // Método con un interfaz como tipo de retorno  
    public Masticable getMasticable( ){  
        return new Chicle( );  
    }  
}
```


Constructores e instanciación

No podemos crear un nuevo objeto sin invocar a su constructor. De hecho, no se puede crear un nuevo objeto sin invocar no sólo el constructor de la clase del objeto, sino además el constructor de **todas y cada una de sus superclases**.

Todas las clases, incluyendo las clases abstract , **DEBEN** tener un constructor. Dos puntos clave acerca de los constructores son que **NO TIENEN TIPO DE RETORNO** y que además sus nombres deben de **coincidir EXACTAMENTE** con el nombre de la clase.

```
class Pelota {
    int tamaño;
    String marca;

    Pelota(String marca, int tamaño) {
        this.marca = marca;
        this.tamaño = tamaño;
    }

    Pelota(String marca) {
        this.marca = marca;
    }

    Pelota(int tamaño) {
        this.tamaño = tamaño;
    } //El constructor puede ser sobrecargado.

    public static void main(String[] args)
    {
        Pelota p1 = new Pelota( "DonBalón",56);
        Pelota p2 = new Pelota( "DoñaPelota");
        Pelota p3 = new Pelota( 90);
        Pelota p4 = new Pelota( ); //¿Compila?
    }
}
```

Constructores en cadena

Supongamos que **Caballo** extiende **Animal** y **Animal** a su vez extiende a **Object**. Veamos que ocurre cuando ejecutamos el código:

```
Caballo = new Caballo ( );
```

1. El constructor de **Caballo** es invocado. Todo constructor invoca al constructor de su superclase con una llamada (implícita) a **super()**, a menos que el constructor invoque a un constructor sobrecargado de la misma clase.

2. El constructor de **Animal** es invocado (El constructor de la superclase).
3. El constructor de **Object** es invocado.
4. Si la clase **Object** tiene alguna variable de instancia, estas deberían tomar valores explícitos (entendemos valores explícitos como valores que son asignados a las variables cuando son declaradas, es decir, **NO valores por defecto**).
5. El constructor de **Object** se completa.
6. Las variables de instancia de **Animal** reciben sus valores explícitos.
7. El constructor de **Animal** se completa.
8. Las variables de instancia de **Caballo** reciben sus valores explícitos.
9. El constructor de **Caballo** se completa.

Reglas para constructores

- Los constructores pueden usar **cualquier modificador de acceso**, incluido **private**. (Un constructor private significa que sólo el código dentro de la clase puede instanciar un objeto de ese tipo, de forma que el constructor private de la clase quiere permitir que se use una instancia de la clase, la clase debe proveer un método static o una variable que permita acceder a la instancia creada dentro de la clase).
- El nombre del constructor debe de coincidir con el nombre de la clase.
- Los constructores nunca tienen tipo de retorno.
- Es legal, pero absurdo, tener un método con el mismo nombre que la clase, pero esto no hace que sea un constructor (además tendrá tipo de retorno).
- Si no escribimos un constructor explícitamente en nuestro código, **el compilador generará uno por defecto**.
- El constructor por defecto es **SIEMPRE** un constructor que **no toma argumentos**.
- Si queremos un constructor sin argumentos y lo escribimos en nuestro código, el compilador ya no proveerá un constructor sin argumentos. Es decir, si escribimos un constructor con argumentos, **ya no tendremos** un constructor sin argumentos a menos que lo escribamos nosotros mismos.

- Todo constructor tiene, como primera sentencia, o una llamada a un constructor sobrecargado (**this()**) de su misma clase o una llamada al constructor de la superclase (**super()**), aunque recordemos que esta llamada puede ser insertada por el compilador.
- Si se crea un constructor explícitamente, y no escribimos en él una llamada a **super()** o una llamada a **this()**, el compilador insertará **una llamada sin argumentos a super()** por nosotros, como primera sentencia dentro del constructor.
- Una llamada a **super()** puede ser una llamada sin argumentos o puede incluir argumentos que son pasados al superconstructor (constructor de la superclase).
- El constructor por defecto es siempre un constructor sin argumentos (Aunque nosotros podemos crear nuestra versión de constructor sin argumentos).
- No podemos hacer una llamada a un método de instancia, o acceder a una variable de instancia hasta después de que se haya ejecutado el superconstructor.
- Sólo las variables y métodos **static** pueden ser accedidos como parte de una llamada a **super()** o **this()**.
- Las clases abstractas tienen también constructores, que son invocados cuando una subclase concreta es instanciada.
- Los interfaces no tienen constructores. Los interfaces no forman parte del árbol de herencia.
- La única manera en la que un constructor puede ser invocado es desde dentro de otro constructor.
- Los constructores NUNCA se heredan, no son métodos. Tampoco pueden ser sobrescritos.

¿Qué aspecto tiene el constructor por defecto?

- ✓ El constructor por defecto tiene el mismo modificador de acceso que la clase.
- ✓ El constructor por defecto no toma argumentos.
- ✓ El constructor por defecto incluye una llamada sin argumentos a **super()**.

¿Qué ocurre si el super constructor tiene argumentos?

Si el constructor de la superclase tiene argumentos, deberemos hacer la llamada a super aportando los argumentos apropiados.

```
class Animal {  
    Animal(String name) { }  
}  
class Caballo extends Animal {  
    Caballo( ) {  
        super( ); // ¿Compila?  
    }  
}
```

Constructores sobrecargados

Sobrecargar un constructor significa escribir múltiples versiones del constructor, cada una con una lista de argumentos diferente.

En el momento que escribimos un constructor para la clase, ya **NO** disponemos del constructor por defecto. Si queremos un constructor sin argumentos para sobrecargar los que hayamos creado, tendremos que escribir uno nosotros mismos.

La sobrecarga de constructores es típicamente utilizada para proveer distintas formas de que los clientes puedan instanciar objetos de nuestra clase.

```
public class Animal{  
    String name;  
    Animal(String name){  
        this.name=name;  
    }  
  
    Animal( ){  
        this.( nombreAleatorio( ));  
    }  
  
    static String nombreAleatorio( ){  
        int x = (int)(Math.random( )*5);  
        String name = new String[ ] { "Fuuffy","Fido","Rover","Spike","Gigi"} [x];  
        // Equivalente a:  
        // String[ ] lista= { "Fuuffy","Fido","Rover","Spike","Gigi"};  
        // String name = lista[x];  
        return name;  
    }  
  
    public static void main(String[ ] args ){  
        Animal a = new Animal( );  
    }  
}
```

```
System.out.println(a.name);
Animal b = new Animal("Zeus");
System.out.println(b.name);
}
}
```

El punto clave está en la línea en negrita. En vez de invocar a **super()**, estamos invocando a **this()**, y **this()** significa llamar a otro constructor en la misma clase. Esto no quiere decir que **super()** no será invocado. Más tarde o más pronto será invocado.

La clave es: La primera línea n un constructor debe ser o una llamada a super() o una llamada a this().

El compilador inserta una llamada a **super()** en cualquier constructor que tenga una llamada explícita al **constructor super**, a menos que el constructor tenga ya una llamada a **this()**.

Bloques de inicialización

Hemos visto dos lugares en los que podemos poner código que lleve a cabo operaciones: métodos y constructores. Los bloques de inicialización son el tercer sitio en un programa Java donde podemos poner código que realice operaciones.

Los bloques static de inicialización se ejecutan cuando la clase es cargada por primera vez, y los bloques de inicialización de instancia se ejecutan cuando sea que se crea una instancia (un poco similar a un constructor).

```
class SmallInt{
    static int x;
    int x;
    static { x=7; } // Bloque de inicialización static
    { y=8; } // Bloque de inicialización de instancia
}
```

Podemos tener varios bloques de inicialización en una clase. Tenemos que tener en cuenta que el orden en que aparecen si tiene importancia: Se ejecutan en el orden en el que se escribieron.

Hay que tener en cuenta las siguientes reglas:

- Los bloques de inicialización se ejecutan en el orden en que aparecen.
- Los bloques de inicialización static se ejecutan **una vez**, cuando la clase es cargada por primera vez.
- Los bloques de inicialización de instancia se ejecutan **cada vez** que una instancia de la clase es creada.

- Los bloques de inicialización de instancia se ejecutan **después** de que el constructor llame a **super()**.

Los bloques de inicialización de instancia se suelen usar para colocar el código que deberían compartir todos los constructores de la clase. Por convenio, los bloques de inicialización se suelen colocar cerca del principio de la definición de la clase.

Statics

Métodos y variables static

La palabra **static** es sinónimo de no perteneciente a ningún objeto concreto, sino pertenencia a la clase.

Las variables y métodos marcados como static pertenecen a la clase, en vez de a alguna instancia en particular. De hecho, podremos usar métodos y variables static sin haber creado ninguna instancia de la clase.

```
class Rana {
    static int contarRanas;// Declarar e inicializar.
    // variable static
    public Rana( )
        contarRanas += 1; // Modificamos el valor en el constructor.
    }
    public static void main (String [ ] args) {
        new Rana( );
        new Rana( );
        new Rana( );
        System.out.println("Ya hay " + contarRanas + " ranas.");
    }
}
```

Si existen instancias creadas de una clase, una variable static de la clase será compartida por todas las instancias de esa clase, por que **SOLAMENTE HAY UNA COPIA**.

```
class Rana {
    static int contarRanas; // Declarar e inicializar.
    // variable static
    public Rana( )
        contarRanas += 1; // Modificamos el valor en el constructor.
    }
}
```

```
class Test{
    public static void main (String [ ] args) {
        Rana rana1 = new Rana( );
    }
}
```

```

Rana1.contarRanas++;
Rana rana2 = new Rana( );
Rana2.contarRanas++;
Rana rana3 = new Rana( );
Rana3.contarRanas++;
System.out.println("Ya hay " + Rana.contarRanas + " ranas.");
}
}

```

La función principal **main** es un método **static** y no está corriendo con ninguna instancia de la clase en particular. Corre en la clase por sí misma. **Un método static no puede acceder a una variable de instancia (no static) porque no hay instancia.** Es decir, no quiere decir que no haya instancias de la clase en la memoria, sino que el método static no sabe nada de ellos. Lo mismo se aplica a los métodos de instancia. Un método static no puede invocar directamente a un método no static (static = class, nonstatic=instance).

Acceso a métodos y variables static

- Un método **static** no puede acceder a una variable de instancia (**no static**), ya que no hay instancia.
- Un método **static** no puede invocar directamente a un método **no static**.
- La manera de acceder a un método o variable static es usar el operador '.' (punto), con el nombre de la clase.
- Java también permite acceder a una variable o método static usando una variable de referencia a un objeto.
- **Un método static NO puede ser sobrescrito.** Esto no significa que no pueda ser redefinido en una subclase, pero redefinir y sobrescribir son dos cosas distintas.

```

class Rana {
    static public saltaLaRana( ){
        System.out.println("Rana de la superclase."); } }

class RanaDeMonte extends Rana{
    static public saltaLaRana( ){
        System.out.println("Rana de la subclase."); } }

public static void main (String [ ] args) {
    Rana rana1= new Rana( );
    RanaDeMonte rdm1= new RanaDeMonte( );
    rana1.saltaLaRana( );
    RanaDeMonte.saltaLaRana( );
} }    //¿Funcionará?

```